

SECURITY & IDENTITY SPECIFICATION

Placement AI (PAI) Authentication Module Architecture

Technical guide: Supabase GoTrue IdP, JWT local validation, Person bootstrap, session cookies, and security gates

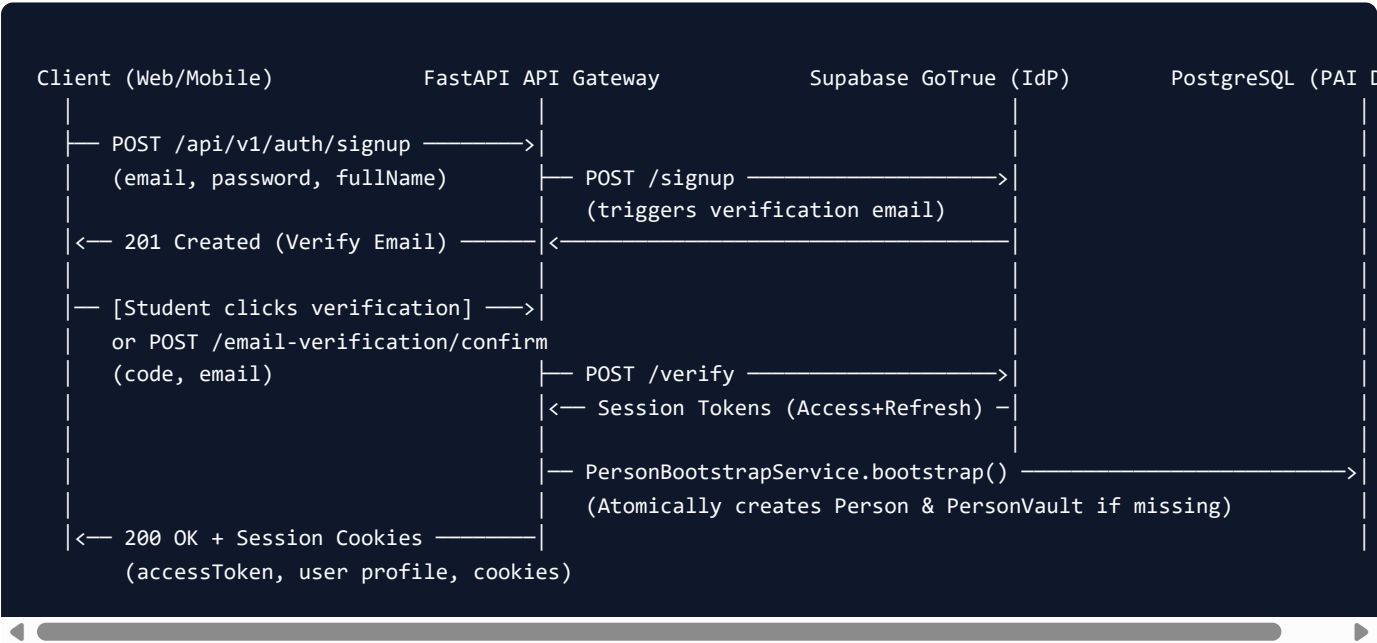
1. Core Architectural Principles

Key Security & Design Invariants:

- **Decoupled Identity vs Profile:** Credentials and passwords reside in Supabase GoTrue. PAI's internal database stores the canonical `Person` and `PersonVault` , lazily provisioned on verified authentication.
- **Local Stateless JWT Verification:** Incoming Bearer tokens are cryptographically checked locally in-memory (symmetric HS256 secret or asymmetric ES256/RS256 via 600s cached JWKS).
- **Double-Submit CSRF + HttpOnly Refresh Tokens:** Access tokens are returned in JSON; refresh tokens reside in `HttpOnly`, `SameSite=Lax` cookies. Sensitive state changes require an matching `X-CSRF-Token` header.
- **Onboarding Gatekeeper:** Authenticated tokens cannot access counseling or document APIs until the student's initial onboarding is marked complete.

2. Authentication Workflows

A. Registration & Email Confirmation Flow



B. Authenticated Request Guard Pipeline

HTTP Request: `Authorization: Bearer `

- 1. Bearer Scheme Check: Strips whitespace, rejects non-Bearer or malformed prefixes.
- 2. JWT Verification (`validate_access_token`):
 - Algorithm detection (HS256 vs ES256/RS256).
 - Signature validation against local secret or cached JWKS (TTL: 600s).
 - Audience validation: ``aud == 'authenticated'``. Expiration: ``exp > now``.
- 3. Distributed Rate Limiter:
 - Consumes 1 token from user request bucket (e.g. 60 requests/min).
- 4. Person Resolution:
 - Queries ``persons`` table where ``external_auth_id == sub``.
- 5. Onboarding Verification:
 - Enforces ``person.onboarding_completed_at`` is not `None``.
 - If `None`, returns 403 `ONBOARDING_INCOMPLETE``.

3. Comprehensive Code & Directory Index

Layer 1: Interfaces & Security Dependencies

File Path	Component Responsibilities & Key Handlers
src/pai/interfaces/api/auth.py	Defines FastAPI authentication and account management endpoints: <code>signup()</code> : Registers user, checks existing identities. <code>login()</code> : Issues session bundle and sets session cookies. <code>refresh_tokens()</code> : Rotates tokens using HttpOnly cookie; guarded by CSRF. <code>logout()</code> : Revokes session on Supabase and clears cookies. <code>confirm_email_verification()</code> : Confirms 6-digit OTP code. <code>change_password()</code> / <code>reset_password()</code> : Password lifecycle. <code>me()</code> : Returns current user identity and onboarding status. <code>delete_account()</code> : Anonymizes local student records and deletes identity from GoTrue.
src/pai/interfaces/api/dependencies.py	Security dependency injection guards: <code>get_current_access_token()</code> : Extracts Bearer token. <code>get_validated_access_token()</code> : Cryptographically validates token and checks rate limits. <code>resolve_person_from_token()</code> : Maps JWT <code>sub</code> to local <code>Person</code> model. <code>require_onboarding_complete()</code> : Enforces onboarding completion. <code>require_csrf()</code> : Validates Double-Submit CSRF cookie pattern against header.

Layer 2: Platform Security & Identity Providers

File Path	Component Responsibilities
src/pai/platform/security/auth/provider.py	Defines the <code>AuthProvider</code> abstract protocol and data models (<code>ProviderUser</code> , <code>ProviderSession</code> , <code>SignupResult</code>).
src/pai/platform/security/auth/supabase.py	Implements <code>AuthProvider</code> via Supabase GoTrue REST API. Contains custom IPv4 transport binding (bypassing Windows IPv6 resolution latency) and error translation mapping.
src/pai/platform/security/auth/jwt.py	Stateless JWT validator. Manages JWKS cache with thread-safe locking, handles HS256/ES256/RS256 algorithms, and validates audience/subject claims.
src/pai/platform/security/auth/service.py	<code>AuthService</code> domain orchestrator. Manages <code>SessionBundle</code> generation (access token, refresh token, CSRF secret creation).
src/pai/platform/security/sensitive_values.py	<code>SensitiveValueCodec</code> : Fernet symmetric encryption engine protecting sensitive fields at rest using <code>VAULT_ENCRYPTION_KEY</code> .

Layer 3: Student Domain & Person Provisioning

File Path	Component Responsibilities
<code>src/pai/domains/student/person/models.py</code>	Defines the student database entities: <code>Person</code> : Local identity record (<code>external_auth_id</code> , <code>email</code> , <code>onboarding_completed_at</code>). <code>PersonVault</code> : Root vault entity storing catalog schema and completion metrics.
<code>src/pai/domains/student/person/service.py</code>	<code>PersonBootstrapService</code> : Executes atomic row-locking provisioning (<code>_get_person_for_update</code>) to link the Supabase user to a local <code>Person</code> and initialize their <code>PersonVault</code> upon verification.

4. Session, Cookie & Security Configuration

Configuration Key	Default / Recommended	Security Purpose
COOKIE_SECURE	false (dev) / true (prod)	Enforces HTTPS-only transmission for session cookies.
COOKIE_SAME_SITE	lax	Defends against Cross-Site Request Forgery (CSRF).
REFRESH_COOKIE_NAME	pai_refresh_token	HttpOnly cookie storing the long-lived refresh token.
CSRF_COOKIE_NAME	pai_csrf_token	Readable cookie for client double-submit token verification.
SUPABASE_JWT_AUDIENCE	authenticated	Audience claim verified against incoming JWTs.
AUTH_HTTP_TIMEOUT_SECONDS	10.0	Network timeout for outbound calls to Supabase GoTrue.

5. Swagger API Quick Reference

Method	Endpoint	Summary	Auth Guard
POST	/api/v1/auth/signup	Register student account	Public
POST	/api/v1/auth/login	Authenticate with email & password	Public
POST	/api/v1/auth/refresh	Rotate access token	Cookie + CSRF
POST	/api/v1/auth/logout	Revoke session	Bearer + CSRF
POST	/api/v1/auth/email-verification/request	Resend confirmation email	Public
POST	/api/v1/auth/email-verification/confirm	Confirm 6-digit OTP	Public
POST	/api/v1/auth/session	Establish session from redirect tokens	Public
POST	/api/v1/auth/password/forgot	Send recovery email	Public
POST	/api/v1/auth/password/reset	Set new password with ticket	Public
POST	/api/v1/auth/password/change	Update password (logged in)	Bearer JWT
GET	/api/v1/auth/me	Get profile & onboarding progress	Bearer JWT
DELETE	/api/v1/account	Anonymize & delete account	Bearer + Cookie